



**KEMENTERIAN PENDIDIKAN
BAHAGIAN MATRIKULASI**

FINAL PROJECT REPORT & PRESENTATION
SEMESTER II (2025/2026)

P48

No.	Matric Number	ID Number
1	MC2516116848	P4801
2	MC2516203254	P4802
3	MC2516116784	P4803
4	MC2516116830	P4804

CANDIDATE'S DECLARATION

We declare that this **FINAL PROJECT REPORT & PRESENTATION** is our own work except for the citations and summaries of which we acknowledged the source.

No.	Matric Number	ID Number	Signature
1	MC2516116848	P4801	
2	MC2516203254	P4802	
3	MC2516116784	P4803	
4	MC2516116830	P4804	

Date: 23/4/2026

TABLE OF CONTENT

1. Introduction.....	3
1.1. Problem Statement	3
1.2. Importance of the Problem.....	4
1.3. Proposed Solution	4
2. Objectives	5
3. Scope of the Project	6
3.1. Scope Of Implementation	6
3.2. Scope Of Capabilities	6
3.3. Limitation of Project	6
4. Methodology	7
4.1. Circuit Diagram	7
4.2. Flowchart	8
5. Specifications.....	14
5.1. Hardware.....	14
5.2. Software	18
6. Implementation	22
6.1. Implementation of The Design Solution.....	22
6.2. Testing approach/Strategy and Design	31
7. Conclusion	34
7.1. Future Work.....	34
8. References.....	36
9. Presentation Video	37
9.1. Timeframe Table.....	37

1. Introduction

The FlowSense Water Availability System is an IoT-based solution developed to address the common issue of inconsistent water availability in shared dormitory toilets. In many college environments, students frequently experience inconvenience due to low water pressure, empty tanks, or undetected leaks, which negatively impact hygiene and daily routines. Existing systems lack real-time monitoring and efficient reporting mechanisms, leading to delayed maintenance responses. To overcome these challenges, this project proposes a smart monitoring system that integrates sensors, a microcontroller, and cloud-based services to detect water levels, identify faults, and provide real-time notifications through a web dashboard and Telegram alerts. By combining automated sensing with user reporting features, the system enhances facility management, improves user convenience, and ensures timely maintenance, making it a practical and effective digital technology solution.

1.1. Problem Statement

We have all known that while pursuing our studies, we are often ought to stay in the dormitories provided, especially for college students. Dormitory life means living together and sharing communal areas such as washing areas and toilets. However, students often face problems while using these communal areas, especially regarding water availability issues. Faced with worn out facilities most colleges have, water availability issues like low water pressure are not an odd amongst students. In this project, we aim to design an IoT-based water availability sensor to help notify students regarding water availability in the toilet stalls.

In this project, the system will use IoT sensors to monitor water availability in the toilet tank by generating a small electric field through its sensing surface. The system will integrate a dashboard to visualize information such as water availability status and the toilet location. If water is not available in the toilet, the system will send a notification to the dashboard to alert the students regarding the status. Other than that, red, green, and yellow LED lights will be installed in front of the toilet stall indicating the water availability status in real time to the students inside the toilet. A push button will be installed inside each stall. In case of the toilet tank itself is broken, the push button enables students to report it to the system. If the toilet has a leak inside, the

system will still be functional as there is a sensor in the toilet tank to sensor trickling water indicating a leakage has happened.

1.2. Importance of the Problem

Water availability plays a crucial role in maintaining hygiene, comfort, and overall well-being, particularly in communal living environments such as college dormitories. In many institutions, students frequently encounter difficulties related to inconsistent water supply or low water pressure, especially in shared toilet facilities. These issues often stem from aging infrastructure and the absence of an effective system to monitor and report water availability in real time. Such circumstances can lead to significant inconvenience, disrupt daily routines, and negatively impact students' quality of life.

The development of an IoT-based water availability monitoring system addresses these challenges by providing a reliable and efficient method to track water conditions within toilet facilities. Through real-time monitoring, notification alerts, and visual indicators, the system enables students to be promptly informed about the status of water availability. Additionally, the integration of a reporting mechanism for faulty toilet tanks enhances the responsiveness of maintenance teams, facilitating timely repairs and minimizing service interruptions.

Overall, this project is significant as it promotes improved facility management, enhances user convenience, and supports sustainable water resource utilization. By implementing modern IoT technology in dormitory settings, the project demonstrates an innovative approach to solving practical problems that affect daily student life and institutional maintenance efficiency.

1.3. Proposed Solution

The proposed system integrates two primary sensors: the XKC-Y25 non-contact water level sensor and the raindrop sensor. The XKC-Y25 sensor will be mounted externally on the toilet tank to detect the water level without direct contact, ensuring durability and ease of installation. Meanwhile, the raindrop sensor will be placed inside the tank to detect trickling water that may indicate internal leakage or malfunction.

Both sensors will be connected to an ESP32 microcontroller, which serves as the main processing and communication unit. The ESP32, housed within a protective PVC enclosure together with a battery, will collect sensor data and transmit it to the Blynk IoT platform via Wi-Fi connectivity. The Blynk application will visualize the data and send real-time notifications to users whenever the water level falls below a predefined minimum threshold.

In addition to mobile notifications, three LED indicators—red, yellow, and green—will be installed above each toilet stall to visually display the water availability status. The green LED signifies that water is available, the red LED indicates that water is unavailable, and the yellow LED represents a faulty or leaking toilet tank. A push button will also be installed inside each stall, enabling users to manually report tank malfunctions or maintenance issues directly to the system.

Through this design, the proposed solution ensures efficient monitoring, timely reporting, and improved maintenance of toilet facilities. It enhances user convenience, promotes proactive maintenance, and contributes to a smarter and more sustainable dormitory environment.

2. Objectives

This project focuses on three key objectives to ensure high quality and better efficiency.

Objective 1 (Identify):

To find and understand the main reasons behind water shortages problem in the college so we can build a strong plan to fix it.

Objective 2 (Develop):

To create and start using a new system that makes our work 90% faster and easier.

Objective 3 (Evaluate):

To check how well the solution works by asking for feedback from the people using it, making sure it meets our long-term goals.

3. Scope of the Project

3.1. Scope Of Implementation

- The project will be implemented in selected male and female dormitory toilets to test the system's real performance.
- The system detects water availability in the toilet tank and displays real-time status through the Blynk app and LED indicators, with notifications for low water or faults.
- The system is designed for college students to check water availability and for maintenance staff to respond quickly to any issues.

3.2. Scope Of Capabilities

- The system can detect the presence of water inside the toilet tank using a non-contact capacitive water level sensor (XKC-Y25).
- The sensor can detect when the water level falls below the minimum threshold and send a digital HIGH or LOW signal to the ESP32 microcontroller
- The ESP32 processes the sensor input and activates LED indicators to show the current water status, such as "water available" or "no water."
- The system is capable of sending real-time water status data to the Blynk IoT platform through a Wi-Fi connection.

3.3. Limitation of Project

- The project will be implemented in selected male and female dormitory toilets to test the system's real performance.
- The system detects water availability in the toilet tank and displays real-time status through the Blynk app and LED indicators, with notifications for low water or faults.
- The system is designed for college students to check water availability and for maintenance staff to respond quickly to any issues.

4. Methodology

4.1. Circuit Diagram

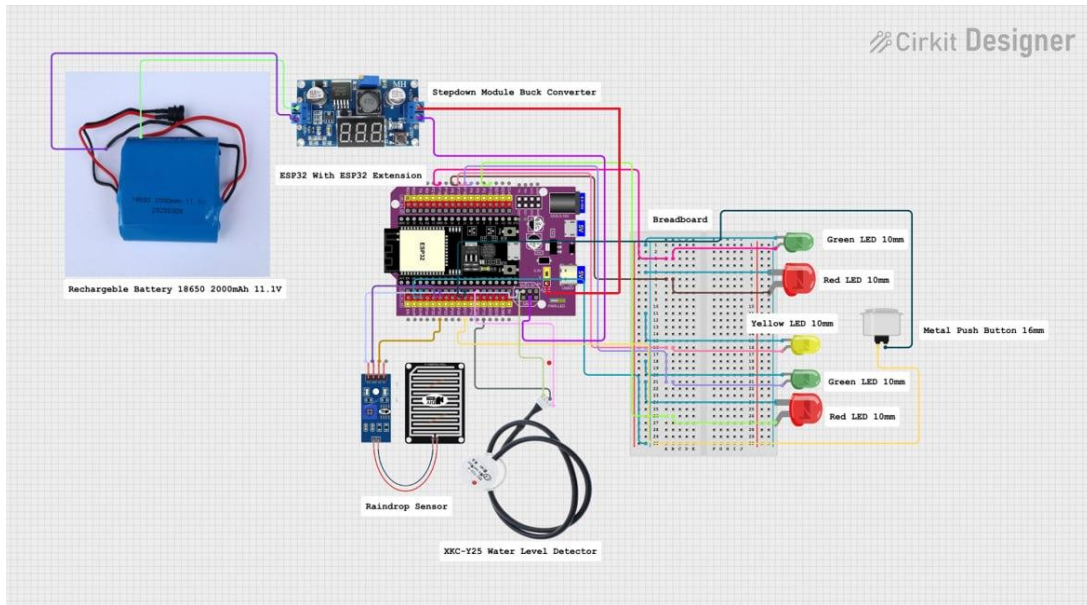


Figure 4.1 Circuit Diagram

Figure 4.1 illustrates the hardware configuration of the FlowSense Water Availability System, showing how the components are connected to support real-time IoT monitoring. As shown in Figure 4.1, the system is powered by a rechargeable 18650 battery connected to a step-down buck converter, which supplies regulated voltage to the ESP32 microcontroller.

The ESP32 acts as the main controller, processing inputs from the raindrop sensor (GPIO 32) and the XKC-Y25 water level sensor (GPIO 13). The system determines water availability based on these sensors, where water is considered available if at least one sensor detects water, and no water if both fail to detect it. A push button connected to GPIO 27 allows users to report issues manually, setting the system status to “BROKEN”.

LEDs are used as output indicators, where green LEDs (GPIO 15 and GPIO 19) indicate water availability, red LEDs (GPIO 4 and GPIO 17) indicate no water, and a yellow LED (GPIO 16) indicates a reported issue. The ESP32 continuously processes

the inputs, updates the system status, and sends data to Firebase while triggering Telegram alerts when necessary.

4.2. Flowchart

Flowcharts are essential tools used in system design and documentation to visually represent the process flow, decision-making logic, and overall workflow of a system. They provide a clear and structured overview of how a system operates, making it easier for developers, engineers, and stakeholders to understand the system behaviour at every stage. In the FlowSense Water Availability System, three distinct flowcharts have been developed to document the system comprehensively: the Hardware Flowchart, the Software/Data Flowchart, and the Integrated System Flowchart. Each flowchart addresses a specific layer of the system, from physical hardware operations to data communication and the combined end-to-end system flow.

4.2.1 Hardware Flowchart

Figure 4.2.1 illustrates the physical operations and logic carried out by the ESP32 microcontroller within the FlowSense Water Availability System. Upon system startup, the ESP32 is initialized and proceeds to continuously read data from three hardware sensors: the Rain Sensor (GPIO 32), the Water Sensor XKC Y25 (GPIO 13), and the Push Button (GPIO 27). The decision-making process begins with a check on the push button; if pressed, the system assigns a status of "BROKEN," activating the Yellow LED (GPIO 16) to indicate a reported fault, and subsequently sends a Telegram alert notifying users of the issue before proceeding to the end of the current loop cycle. If the button is not pressed, the system evaluates whether the rain sensor or water sensor detects the presence of water. If water is detected, the status is set to "AVAILABLE" and the Green LEDs (GPIO 15 and 19) are activated, accompanied by a conditional Telegram notification if the status has changed. Conversely, if no water is detected, the status is set to "NO WATER" and the Red LEDs (GPIO 4 and 17) are turned on, with a corresponding Telegram alert sent to notify users. After completing all status evaluations and LED activations, the system loops back to read the sensors again, ensuring continuous and real-time hardware monitoring.

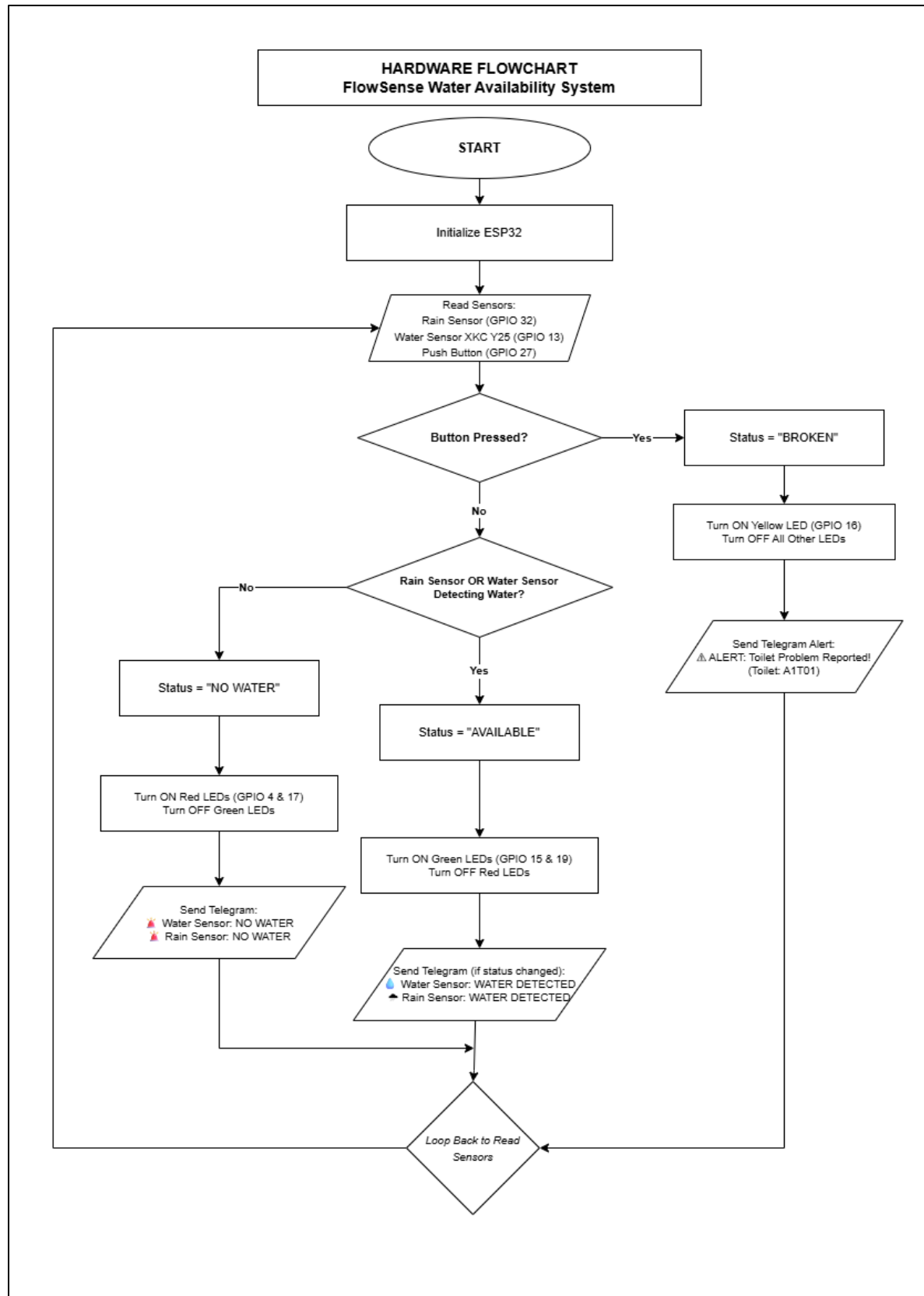


Figure 4.2.1 Hardware Flowchart

4.2.2 Software/Data Flowchart

Figure 4.2.2 describes the data processing and communication logic of the FlowSense Water Availability System, focusing on how information is transmitted from the ESP32 microcontroller to the Firebase Realtime Database and subsequently delivered to end users. Upon system start, the ESP32 attempts to establish a WiFi connection. If the connection fails, the system automatically retries until a stable connection is achieved. Once connected, the system enters its main operational loop, reading sensor data from the ESP32 at regular intervals. Based on the sensor readings, the system determines the current water availability status using a predefined logic: if the push button is pressed, the status is set to "BROKEN"; if rain or water is detected, the status is set to "AVAILABLE"; otherwise, the status defaults to "NO WATER." The determined status, along with other relevant fields such as toilet_code, block, floor, toilet_num, raindrop_sensor, xkc_sensor, button_pressed, esp_id, and last_update, is transmitted to Firebase via a PATCH HTTP request every five seconds. The Firebase Realtime Database is updated accordingly, and the system website retrieves and displays the real-time status to users. Additionally, a Telegram bot is configured to send automated alerts, but only when a change in status is detected, thereby preventing redundant notifications. The system then loops back continuously to maintain uninterrupted monitoring and data synchronization.

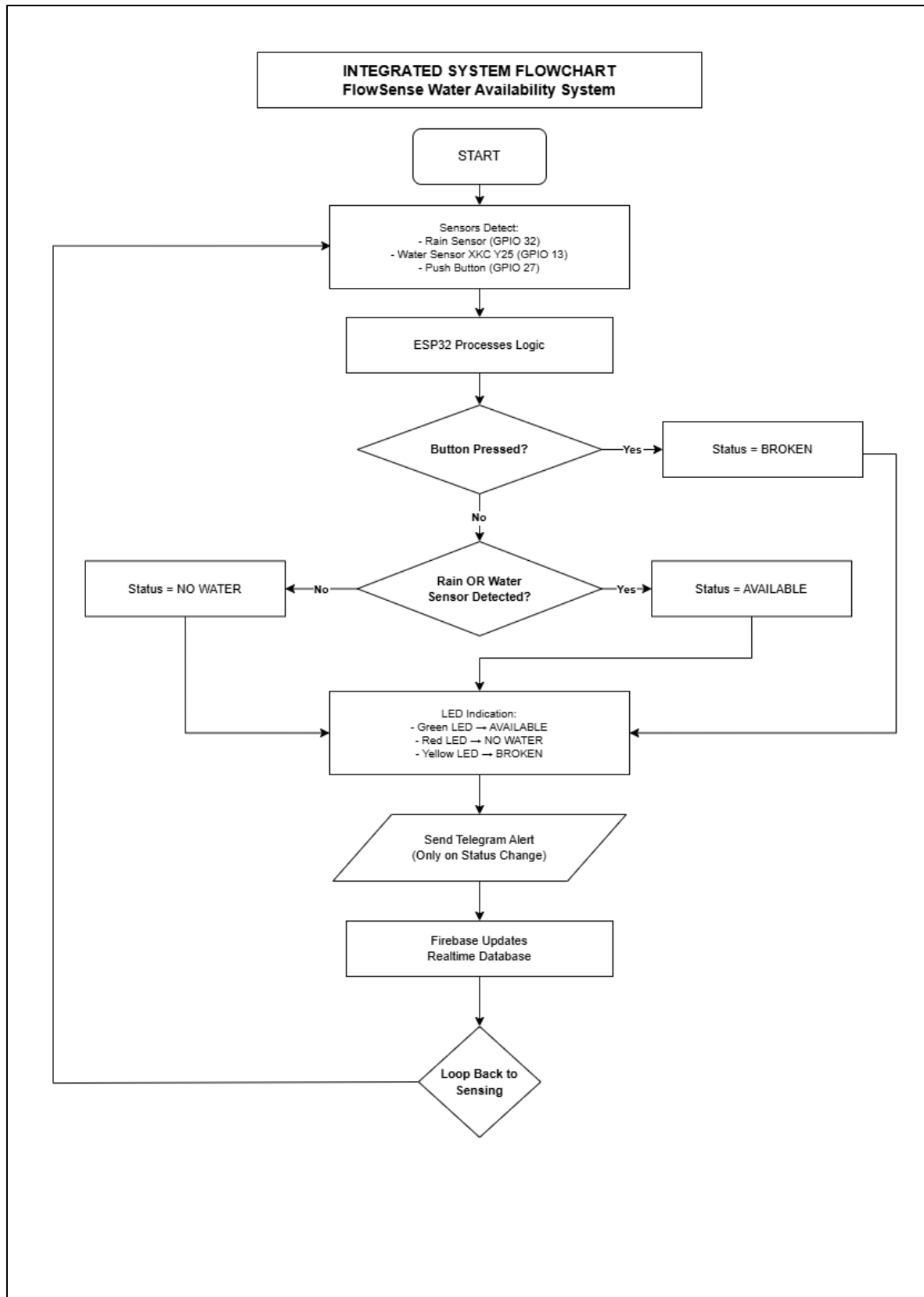


Figure 4.2.2 Software / Data Flowchart

4.2.3 Integrated System Flowchart

Figure 4.2.3 presents a holistic view of the FlowSense Water Availability System, combining both the hardware and software layers into a single, unified process flow. The flowchart begins with sensor detection, where the Rain Sensor (GPIO 32), Water Sensor XKC Y25 (GPIO 13), and Push Button (GPIO 27) simultaneously collect environmental data. The collected data is forwarded to the ESP32, which processes the logic and applies a two-tier decision tree. First, if the push button is pressed, the system registers a "BROKEN" status. If not, it checks whether the rain or water sensor has detected water; a positive detection results in an "AVAILABLE" status, while a negative result assigns a "NO WATER" status. Based on the determined status, the appropriate LED indication is activated: Green LED for AVAILABLE, Red LED for NO WATER, and Yellow LED for BROKEN. A Telegram alert is dispatched only when the status changes, ensuring minimal notification overhead. The ESP32 then transmits the processed data to Firebase via a PATCH request, updating the Realtime Database. The system website retrieves this data and displays the live water availability status to users, enabling real-time remote monitoring. Upon completing the data transmission and display cycle, the system loops back to the sensor reading phase, maintaining continuous and automated operation throughout the deployment period.

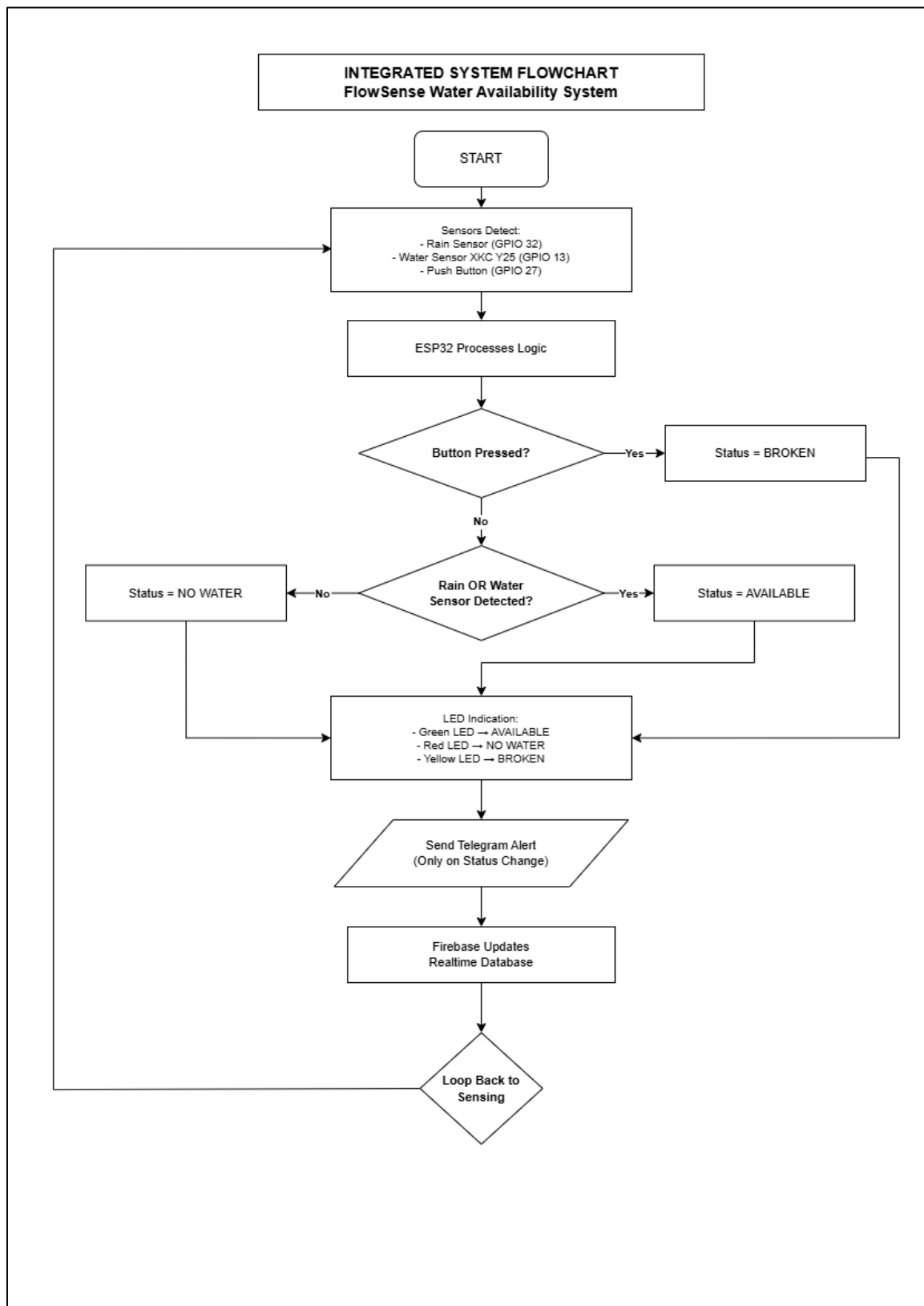
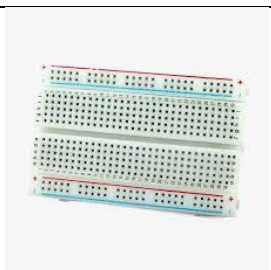


Figure 4.2.3 Integrated System Flowchart

5. Specifications

List of Components & Specifications, Functions of Components and Pictures of Components

5.1. Hardware

	COMPONENT NAME	SPECIFICATION	FUNCTION	COMPONENT PICTURE
1	ESP32	Microcontroller with Wi-Fi and Bluetooth	Acts as the main controller to process data and control other components.	
2	XKC-Y25 Capacitive Water Level Sensor (0.5 m)	Non-contact type sensor	Detects the presence or absence of water on specific minimum level in a container without direct contact.	
3	Expansion Board - ESP32-S	Compatible base board for ESP32	Provides easy connections for ESP32 pins and external components.	
4	Small Solderless breadboard	170 tie-points	Used to build circuits without soldering, allows testing and connection of components.	

5	LM2596 Step Down (Buck) Converter module	Input voltage of 3V-40V DC, adjustable output voltage of 1.5V-35V, maximum current 3A	To convert a higher DC input voltage into a lower, adjustable DC output voltage.	
6	USB Cable		Supplies power and uploads code to the ESP32.	
7	Lithium Battery 18650	2000mAh	Provides portable power supply to the system.	
8	Battery Holder 18650 2 Slot	Holds 2 18650 lithium batteries	Secures and connects batteries to the circuit.	
9	Raindrop Sensor	Analog type sensor	Detects the presence of rain or water drops on its surface.	
10	PVC Box Electrical 8×6×3	Plastic enclosure	Protects circuit and component from dust and moisture.	

11	Metal No Lock Push Button	16mm	Allows manual input to trigger or reset certain actions in the system.	
12	Heat Shrink Tube Wire	5mm	Insulate and protect electrical wires and connections from damage, short circuits, and moisture.	
11	Green, Yellow, Red LED	10mm	Indicates the water availability status in the toilet stall	
12	Jumper Wire		Connect electric components together to allow electric current flows through.	
13	Clear storage box 10L	30cm x 21cm x 17cm	Stores water, acting like a toilet tank.	

14	Water dispenser nozzle		Allows water in the container to be emptied out	
----	------------------------	--	---	---

5.2. Software

The FlowSense Water Availability System is a real-time IoT-based monitoring and management system designed to facilitate continuous observation of water flow and availability. The system integrates three core software layers: an embedded system running on the ESP32 microcontroller, a cloud-based backend powered by Firebase, and a web-based frontend dashboard accessible through a standard web browser. Together, these components form a seamlessly interconnected platform that enables automated data acquisition, cloud-based data management, real-time alert delivery, and user-friendly interface for system monitoring and administration.

5.2.1 Embedded System Software

Table 5.2.1: Embedded System Software Tools

Software / Tool	Version	Function	Description
Arduino IDE	2.3.x	Development Environment	Integrated development environment used to write, compile, and upload firmware code to the ESP32 microcontroller.
ESP32 Board Package	2.0.x	Hardware Abstraction	Provides the necessary board definitions, pin configurations, and hardware abstraction libraries for programming the ESP32.
WiFi Library	Built-in	Wireless Connectivity	Enables the ESP32 to connect to a local WiFi network for transmitting sensor data to the cloud backend.

Software / Tool	Version	Function	Description
HTTPClient Library	Built-in	HTTP Communication	Facilitates HTTP and HTTPS requests from the ESP32 to Firebase REST API endpoints for data read and write operations.
ArduinoJson Library	6.x	Data Serialization	Handles serialization and deserialization of JSON data structures used during communication between the ESP32 and Firebase.
UniversalTelegramBot Library	1.3.x	Telegram Alert Integration	Enables the ESP32 to send automated alert messages via the Telegram Bot API whenever critical water availability thresholds are detected.

The embedded software deployed on the ESP32 is responsible for acquiring and processing sensor readings related to water level and flow rate. It applies threshold-based status determination logic to classify the current water availability as normal, low, or critical. Once processed, the sensor data is serialized into JSON format and transmitted to Firebase Realtime Database via HTTPS using the HTTPClient library. In parallel, the UniversalTelegramBot library is used to dispatch real-time alert messages to designated recipients through the Telegram messaging platform whenever abnormal water conditions are detected.

5.2.2 Backend / Cloud Services

Table 5.2.2: Backend and Cloud Service Tools

Software / Tool	Type	Function	Description
Firebase Realtime Database	NoSQL Cloud Database	Real-Time Data Storage	Stores and synchronises sensor data in real time across all connected clients, enabling immediate reflection of changes on the web dashboard.
Firebase Authentication	Identity Service	User Access Control	Manages user registration and authentication to ensure that only authorised administrators can access and control the system through the web interface.
Firebase REST API	Web API	Device-Cloud Communication	Provides HTTP-based endpoints through which the ESP32 microcontroller communicates with the Firebase backend to push sensor data and retrieve configuration parameters.

Firebase serves as the cloud backbone of the FlowSense system, providing persistent and real-time data storage through its Realtime Database service. All sensor readings transmitted from the ESP32 are stored in the Firebase database, where they are immediately available to all connected clients without requiring manual refresh. Firebase Authentication enforces access control by verifying user credentials before granting access to the administrative web interface. The Firebase REST API acts as the communication bridge between the ESP32 microcontroller and the cloud, enabling reliable and secure data exchange over the internet.

5.2.3 Frontend Application

Table 5.2.3: Frontend Application Technologies

Software / Tool	Language	Function	Description
HTML	Markup Language	Page Structure	Defines the structural layout and semantic elements of the web-based dashboard, including data display panels, navigation menus, and control forms.
CSS	Stylesheet Language	Visual Styling	Governs the visual presentation of the dashboard, including typography, layout, colour scheme, and responsiveness across different screen sizes.
JavaScript	Scripting Language	Client-Side Logic	Implements interactive behaviour on the dashboard, including real-time data updates, event handling, and dynamic rendering of sensor readings and status indicators.
Firebase SDK (Web)	JavaScript SDK	Firebase Integration	Enables the frontend application to communicate directly with Firebase Realtime Database and Authentication services for live data retrieval and user session management.

The frontend application provides a web-based real-time dashboard through which administrators can monitor the current water availability status of the system. Built using standard web technologies — HTML, CSS, and JavaScript — the dashboard renders sensor data dynamically by subscribing to live updates from Firebase Realtime Database via the Firebase Web SDK. The monitoring interface displays key metrics

including water level readings, flow rate, and system status, and updates automatically whenever new data is received from the ESP32. Administrative control features allow authorised users to configure system thresholds, manage user accounts, and review historical sensor data, all within a secure and authenticated web session.

5.2.4 System Integration Overview

The FlowSense Water Availability System operates through a three-tier integration architecture that ensures continuous, real-time data flow from the physical environment to the end user. At the device layer, the ESP32 microcontroller periodically acquires sensor data, processes it, and transmits the readings to Firebase Realtime Database via the Firebase REST API over a secured WiFi connection. At the cloud layer, Firebase acts as the centralised data repository, instantly propagating any changes to all subscribed clients and enforcing authentication for administrative access. At the presentation layer, the web-based dashboard retrieves live data from Firebase using the Web SDK and renders it in real time, providing administrators with an up-to-date view of the system without requiring manual page refreshes. In parallel with the data pipeline, the Telegram alert subsystem operates as an independent notification channel; when the ESP32 detects that water availability has fallen below predefined critical thresholds, it dispatches automated alert messages directly to the designated Telegram contacts using the UniversalTelegramBot library. This integrated architecture ensures that system stakeholders are informed promptly of any water availability issues, regardless of whether they are actively monitoring the web dashboard.

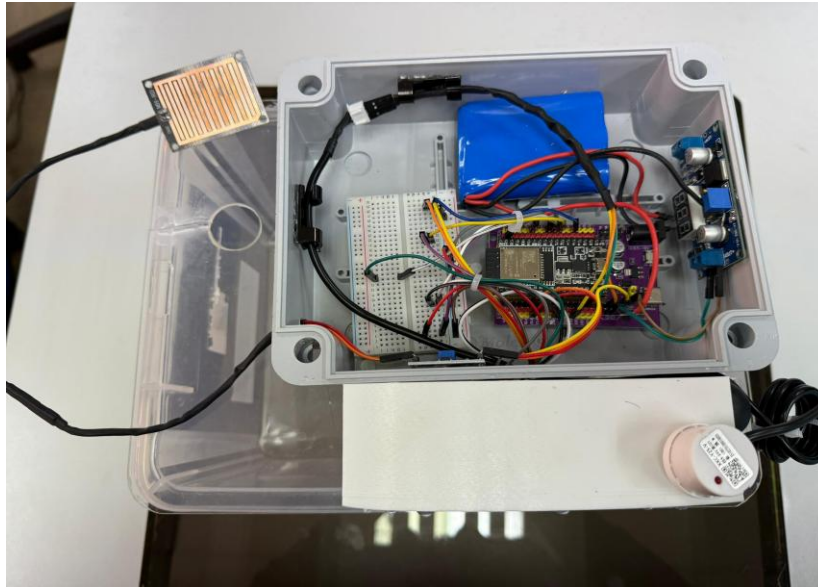
6. Implementation

6.1. Implementation of The Design Solution

The implementation phase of the FlowSense Water Availability System involved integrating hardware components, embedded programming, and cloud-based communication technologies to develop a complete Internet of Things (IoT) monitoring system. The system was implemented based on the previously designed circuit diagram,

system architecture, and operational flowchart to ensure that all components function according to the intended design.

From the hardware perspective, the system uses an ESP32 as the central controller that reads sensor data, processes decision logic, and communicates with the cloud platform. Several sensors were integrated into the system to monitor water availability and detect abnormal conditions. The XKC-Y25 non-contact water level sensor is used to detect the presence of water in the pipe without direct contact. This sensor helps determine whether water is flowing through the system. In addition, a raindrop sensor is installed to detect potential water leakage or dripping conditions that may occur in the toilet system. A push button is also implemented as a manual reporting mechanism. This button allows users to report a faulty or malfunctioning toilet system by triggering a fault condition in the system. All components were connected according to the designed circuit using jumper wires on a breadboard. The system is powered by a rechargeable 18650 lithium battery, while a DC buck converter is used to regulate and stabilise the voltage supplied to the ESP32 and sensors. To provide visual feedback regarding system status, three LED indicators are used which are red, green and yellow. The XKC-Y25 water level sensor and the raindrop sensor operate independently to provide more accurate water condition detection. Each sensor controls its own pair of LED indicators, consisting of a green LED and a red LED. The XKC-Y25 sensor is responsible for detecting whether the water level has reached the minimum threshold inside the pipe. Meanwhile, the raindrop sensor detects the presence of water that may indicate leakage or dripping conditions. By combining the outputs from both sensors, the system can differentiate between normal water supply conditions, absence of water, and possible leakage scenarios. The yellow LED was used to indicate a fault or broken condition which will light up when the user push the push button.



From the software perspective, the ESP32 was programmed using the Arduino Integrated Development Environment (Arduino IDE). The program continuously reads sensor data and processes it according to the decision-making logic defined in the system flowchart. The system determines three main operating conditions based on sensor reading and use interaction:

- AVAILABLE - Water is detected by the sensors
- NO WATER - No water is detected by the sensors
- BROKEN - The push button is pressed to report a malfunction

Once the condition is determined, the ESP32 activates the corresponding LED indicator and transmits the system status to the cloud platform via WiFi.

The following code represents the finalized firmware for the FlowSense system. It integrates all sensor logic, WiFi connectivity, and cloud synchronization protocols to ensure the system operates reliably in a real-world environment.

```

1  ✓ /*****
2  FlowSense - Firebase + Telegram
3  ESP32 v4 - Hysteresis Edition
4
5  ✓ Sensor pins:
6  RAIN_SENSOR → GPIO 32 (ANALOG)
7  WATER_SENSOR → GPIO 13 (DIGITAL - XKC)
8  ALERT_BUTTON → GPIO 27
9
10 ✓ LED pins:
11 RAIN_LED_GREEN → GPIO 15
12 RAIN_LED_RED → GPIO 4
13 WATER_LED_GREEN → GPIO 19
14 WATER_LED_RED → GPIO 17
15 ALERT_LED → GPIO 16
16
17 ✓ RAIN_SENSOR:
18 - Direct analogRead, no delay
19 - Hysteresis: dua threshold untuk elak Telegram spam
20   - Bawah 2800 = ada air
21   - Atas 3200 = tiada air
22   - Antara 2800-3200 = kekal status lama
23
24 ✓ BROKEN FLAG LOGIC:
25 - User tekan button sekejap → brokeFlag = true → status kekal "broken"
26 - Status kekal broken walaupun button dilepas / sensor detect air / ESP32 reboot
27 - Hanya admin boleh clear dari website → broken_confirmed = false di Firebase
28 - ESP32 check Firebase setiap 5 saat untuk detect admin clear
29 *****/
30
31 #include <WiFi.h>
32 #include <HTTPClient.h>
33 #include <WiFiClientSecure.h>
34 #include <UniversalTelegramBot.h>
35 #include <ArduinoJson.h>
36
37 // ===== WIFI =====
38 char ssid[] = "HONOR X9c";
39 char pass[] = "Apizganteng";
40
41 // ===== TELEGRAM =====
42 #define BOTtoken "9709513438:AAGxHP1UjwFwX-Csiw9VsrDc64qax1lYalk"
43 #define CHAT_ID "7602717658"
44
45 // ===== FIREBASE =====
46 #define FIREBASE_HOST "https://flowsense-db763-default-rtdb.asia-southeast1.firebaseio.com/"
47 #define FIREBASE_AUTH "hdwKjhtwFZbpIDrH2q5TUDIMjP45vL0JH2x52RD"
48
49 // ===== TOILET INFO =====
50 #define TOILET_CODE "A1T01"
51 #define ESP_ID "ESP32-FlowSense"
52 #define BLOCK "A"
53 #define FLOOR_NUM 1
54 #define TOILET_NUM 1
55
56 // ===== PIN =====
57 #define RAIN_SENSOR 32 // ADC1 - analog read
58 #define RAIN_LED_GREEN 15
59 #define RAIN_LED_RED 4
60 #define WATER_SENSOR 13 // XKC - digital read
61 #define WATER_LED_GREEN 19
62 #define WATER_LED_RED 17
63 #define ALERT_BUTTON 27
64 #define ALERT_LED 16
65
66 // ===== HYSTERESIS THRESHOLD =====
67 // Turun bawah 2800 → ada air
68 // Naik atas 3200 → tiada air
69 // Antara 2800-3200 → kekal status lama (elak spam)
70 #define RAIN_THRESHOLD_WET 2800
71
72 #define RAIN_THRESHOLD_DRY 3200
73
74 // ===== GLOBALS =====
75 WiFiClientSecure client;
76 UniversalTelegramBot bot(BOTtoken, client);
77
78 int rainStatus = -1; // -1=unknown, 0=dry, 1=wet
79 int waterStatus = -1;
80
81 // ===== BROKEN FLAG =====
82 bool brokeFlag = false;
83 bool lastButtonState = false;
84 bool brokeReported = false;
85
86 // ===== TIMING =====
87 String lastFirebaseStatus = "";
88 unsigned long lastSendTime = 0;
89 unsigned long lastFirebaseCheck = 0;
90 #define SEND_INTERVAL 1000
91 #define FIREBASE_CHECK_INTERVAL 5000
92
93 // ===== READ RAIN SENSOR - direct, no delay =====
94 int readRainSensor() {
95   return analogRead(RAIN_SENSOR);
96 }
97
98 // ===== RAIN HYSTERESIS =====
99 // Elak flip-flop / spam bila nilai kat sempadan threshold
100 bool getRainIsWater(int rawValue) {
101   if (rawValue < RAIN_THRESHOLD_WET) return true; // confirm ada air
102   else if (rawValue > RAIN_THRESHOLD_DRY) return false; // confirm tiada air
103   else return (rainStatus == 1); // kekal status lama
104 }
105
106 // ===== DETERMINE STATUS =====

```

```

106 String determineStatus(bool rainHasWater, bool waterHasWater) {
107     if (brokeFlag) return "broken";
108     if (rainHasWater && waterHasWater) return "available";
109     return "no-water";
110 }
111
112 // ===== SEND TO FIREBASE =====
113 void sendToFirebase(bool rain, bool water, String status) {
114     if (WiFi.status() != WL_CONNECTED) return;
115
116     HTTPClient http;
117
118     String url = String(FIREBASE_HOST) + "/toilets/" + TOILET_CODE + ".json";
119     if (strlen(FIREBASE_AUTH) > 0) {
120         url += "?auth=" + String(FIREBASE_AUTH);
121     }
122
123     StaticJsonDocument<300> doc;
124     doc["toilet_code"] = TOILET_CODE;
125     doc["block"] = BLOCK;
126     doc["floor"] = FLOOR_NUM;
127     doc["toilet_num"] = TOILET_NUM;
128     doc["raindrop_sensor"] = rain;
129     doc["xkc_sensor"] = water;
130     doc["button_pressed"] = brokeFlag;
131     doc["status"] = status;
132     doc["broken_confirmed"] = brokeFlag;
133     doc["esp_id"] = ESP_ID;
134     doc["last_update"] = millis();
135
136     String payload;
137     serializeJson(doc, payload);
138
139     http.begin(url);
140     http.addHeader("Content-Type", "application/json");
141
142     int code = http.PATCH(payload);
143
144     if (code == 200 || code == 204) {
145         Serial.println("✅ Firebase updated! Status: " + status);
146     } else {
147         Serial.println("❌ Firebase FAILED. HTTP: " + String(code));
148         Serial.println(http.getString());
149     }
150     http.end();
151 }
152
153 // ===== CHECK FIREBASE - admin clear broken =====
154 void checkFirebaseForAdminClear() {
155     if (WiFi.status() != WL_CONNECTED) return;
156     if (!brokeFlag) return;
157
158     HTTPClient http;
159
160     String url = String(FIREBASE_HOST) + "/toilets/" + TOILET_CODE + "/broken_confirmed.json";
161     if (strlen(FIREBASE_AUTH) > 0) {
162         url += "?auth=" + String(FIREBASE_AUTH);
163     }
164
165     http.begin(url);
166     int code = http.GET();
167
168     if (code == 200) {
169         String response = http.getString();
170         response.trim();
171         Serial.println("🔍 broken_confirmed = " + response);
172
173         if (response == "false") {
174             Serial.println("✅ Admin cleared broken flag!");
175             brokeFlag = false;
176
177             brokeReported = false;
178             digitalWrite(ALERT_LED, LOW);
179
180             bot.sendMessage(CHAT_ID,
181                 "✅ Toilet " + String(TOILET_CODE) + " telah ditandakan FIXED oleh admin.\n"
182                 "🔄 Sistem kembali monitor sensor.");
183         }
184
185         int rainRaw = readRainSensor();
186         bool rain = getRainHasWater(rainRaw);
187         bool water = (digitalRead(WATER_SENSOR) == LOW);
188         String newStatus = determineStatus(rain, water);
189         sendToFirebase(rain, water, newStatus);
190         lastFirebaseStatus = newStatus;
191     } else {
192         Serial.println("⚠️ Firebase GET failed. HTTP: " + String(code));
193     }
194
195     http.end();
196 }
197
198 // ===== CHECK SYSTEM =====
199 void checkSystem() {
200     // ===== RAIN SENSOR - direct read + hysteresis =====
201     int rainRaw = readRainSensor();
202     bool rainHasWater = getRainHasWater(rainRaw);
203
204     Serial.println("🌧️ RAIN RAW: " + String(rainRaw) +
205         " | hasWater: " + String(rainHasWater ? "YES ✅" : "NO ❌"));
206
207     // ===== XKC SENSOR - digital =====
208     bool waterHasWater = (digitalRead(WATER_SENSOR) == LOW);

```

```

246     if (rainStatus != 1) {
247         bot.sendMessage(CHAT_ID, "☔ Rain Sensor: WATER DETECTED (Toilet: " + String(TOILET_CODE) + ")");
248         rainStatus = 1;
249     }
250 } else {
251     digitalWrite(RAIN_LED_GREEN, LOW);
252     digitalWrite(RAIN_LED_RED, HIGH);
253     if (rainStatus != 0) {
254         bot.sendMessage(CHAT_ID, "🔴 Rain Sensor: NO WATER (Toilet: " + String(TOILET_CODE) + ")");
255         rainStatus = 0;
256     }
257 }
258
259 // ===== WATER LED + TELEGRAM (XKC) =====
260 if (waterHasWater) {
261     digitalWrite(WATER_LED_GREEN, HIGH);
262     digitalWrite(WATER_LED_RED, LOW);
263     if (waterStatus != 1) {
264         bot.sendMessage(CHAT_ID, "💧 Water Sensor: WATER DETECTED (Toilet: " + String(TOILET_CODE) + ")");
265         waterStatus = 1;
266     }
267 } else {
268     digitalWrite(WATER_LED_GREEN, LOW);
269     digitalWrite(WATER_LED_RED, HIGH);
270     if (waterStatus != 0) {
271         bot.sendMessage(CHAT_ID, "🔴 Water Sensor: NO WATER (Toilet: " + String(TOILET_CODE) + ")");
272         waterStatus = 0;
273     }
274 }
275
276 // ===== STATUS & FIREBASE =====
277 String status = determineStatus(rainHasWater, waterHasWater);
278
279 if (status != lastFirebaseStatus) {
280     Serial.println("\n📡 STATUS CHANGED: " + lastFirebaseStatus + " → " + status);
281     sendToFirebase(rainHasWater, waterHasWater, status);
282     lastFirebaseStatus = status;
283     lastSendTime = millis();
284 }
285 else if (millis() - lastSendTime >= SEND_INTERVAL) {
286     sendToFirebase(rainHasWater, waterHasWater, status);
287     lastSendTime = millis();
288 }
289 }
290
291 // ===== SETUP =====
292 void setup() {
293     Serial.begin(115200);
294     delay(500);
295     Serial.println("\n=====");
296     Serial.println(" FlowSense - Firebase + Telegram v4");
297     Serial.println(" Hysteresis Edition - No Delay");
298     Serial.println("=====");
299
300     pinMode(RAIN_LED_GREEN, OUTPUT);
301     pinMode(RAIN_LED_RED, OUTPUT);
302     pinMode(WATER_SENSOR, INPUT);
303     pinMode(WATER_LED_GREEN, OUTPUT);
304     pinMode(WATER_LED_RED, OUTPUT);
305     pinMode(ALERT_BUTTON, INPUT_PULLUP);
306     pinMode(ALERT_LED, OUTPUT);
307     // RAIN_SENSOR (GPIO32) - no pinMode needed
308
309     // Connect WiFi
310     Serial.print("Connecting to WiFi: ");
311     Serial.println(ssid);
312     WiFi.begin(ssid, pass);
313
314     int attempts = 0;
315     while (WiFi.status() != WL_CONNECTED && attempts < 20) {
316         delay(500);
317         Serial.print(".");
318         attempts++;
319     }
320
321     if (WiFi.status() == WL_CONNECTED) {
322         Serial.println("\n📶 WiFi Connected!");
323         Serial.println("IP: " + WiFi.localIP().toString());
324     } else {
325         Serial.println("\n❌ WiFi GAGAL!");
326     }
327
328     client.setInsecure();
329
330     // ===== CHECK FIREBASE ON BOOT - recover brokeFlag =====
331     Serial.println("🔍 Checking Firebase for previous broken state...");
332     HTTPClient http;
333     String url = String(FIREBASE_HOST) + "/toilets/" + TOILET_CODE + "/broken_confirmed.json";
334     if (strlen(FIREBASE_AUTH) > 0) url += "?auth=" + String(FIREBASE_AUTH);
335     http.begin(url);
336     int code = http.GET();
337     if (code == 200) {
338         String response = http.getString();
339         response.trim();
340         Serial.println("broken_confirmed on boot = " + response);
341         if (response == "true") {
342             brokeFlag = true;
343             brokeReported = true;
344             digitalWrite(ALERT_LED, HIGH);
345             Serial.println("🔴 Recovered broken state from Firebase!");
346         }
347     }
348     http.end();
349
350     // Boot test rain sensor

```

```

351 Serial.println("\n 🌧️ BOOT TEST Rain Sensor:");
352 int bootRaw = readRainSensor();
353 Serial.println("   RAW: " + String(bootRaw) +
354               (bootRaw < RAIN_THRESHOLD_WET ? " → ADA AIR 🟢 " :
355               bootRaw > RAIN_THRESHOLD_DRY ? " → TIADA AIR 🔴 " :
356               " → ZONE TENGAH (kekal status lama)"));
357
358 bot.sendMessage(CHAT_ID,
359               "🟢 FlowSense Online!\n"
360               "📶 Firebase + Telegram Active\n"
361               "🌧️ Rain Sensor: ANALOG | Wet<" + String(RAIN_THRESHOLD_WET) +
362               " | Dry>" + String(RAIN_THRESHOLD_DRY) + "\n"
363               "🚽 Monitoring: " + String(TOILET_CODE) + "\n" +
364               (brokeFlag ? "🔥 Status: MASIH BROKEN (belum di-fixed)" : "🟢 Status: Normal")
365               );
366
367 lastSendTime = 0;
368 }
369
370 // ===== LOOP =====
371 void loop() {
372   static unsigned long lastCheck = 0;
373   if (millis() - lastCheck >= 1000) {
374     checkSystem();
375     lastCheck = millis();
376   }
377
378   if (millis() - lastFirebaseCheck >= FIREBASE_CHECK_INTERVAL) {
379     checkFirebaseForAdminClear();
380     lastFirebaseCheck = millis();
381   }
382
383   // Reconnect wifi kalau putus
384   if (WiFi.status() != WL_CONNECTED) {
385     Serial.println("Wifi putus, reconnecting...");
386
387     WiFi.begin(ssid, pass);
388     delay(3000);
389   }
390 }

```

Figure 6.1.1 ESP-32 Hardware Code

For cloud integration, the system utilises the Firebase Realtime Database to store and synchronise data in real time. Communication between the ESP32 and Firebase is performed using HTTP requests, where the system sends data in JSON format to ensure efficient data exchange.

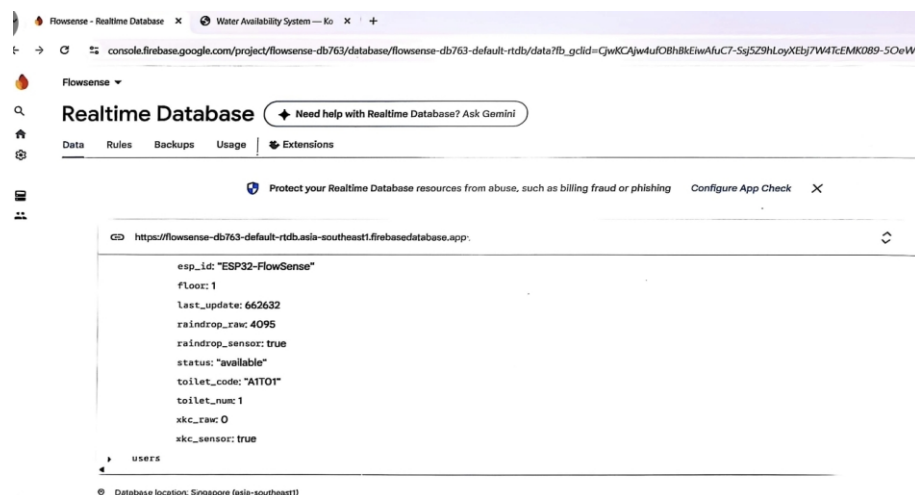


Figure 6.1.2 Firebase Real-time Database

To enhance the responsiveness of the system, Telegram Bot integration was implemented. This feature enables the system to automatically send alert notifications whenever abnormal conditions are detected, such as when there is no water supply or when a system fault occurs.



Figure 6.1.3 Telegram interfaces

In addition, a web-based dashboard was developed using HTML, CSS, and JavaScript to allow users to monitor the water availability status remotely. The dashboard retrieves real-time data from Firebase using the Firebase Web SDK, ensuring that system information is updated automatically without requiring manual refresh.

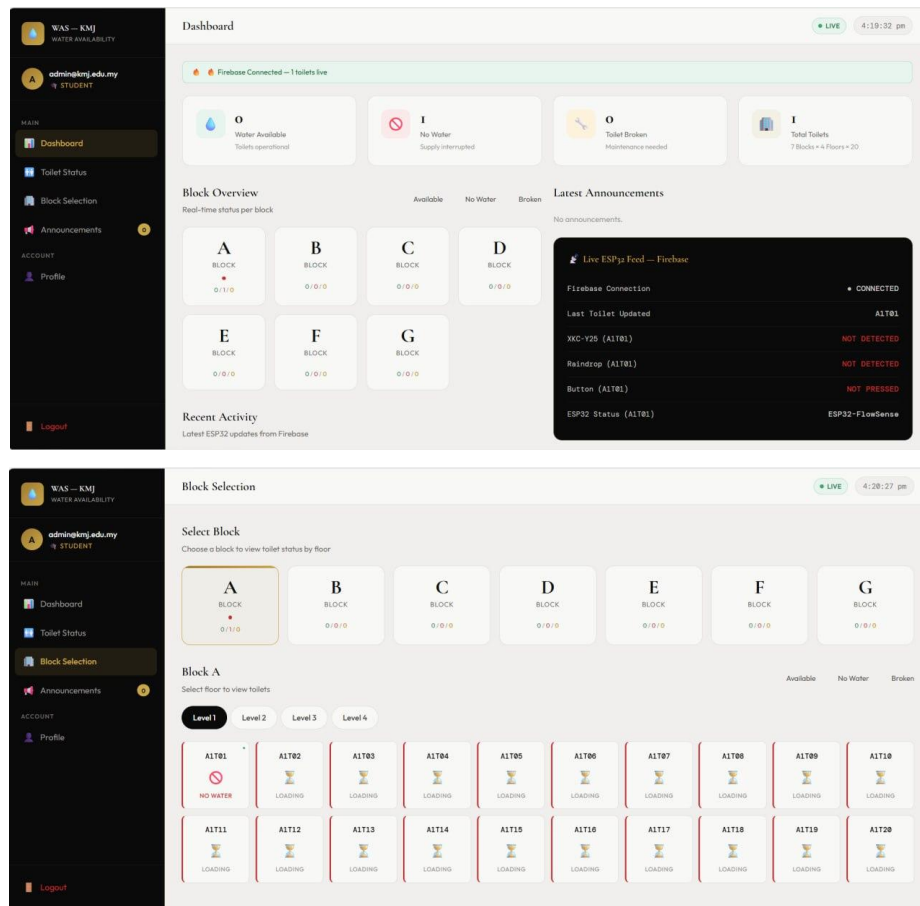


Figure 6.1.4 web-based dashboard that monitors water availability in real-time using cloud data synchronization

Overall, the FlowSense system implementation follows a three-layer IoT architecture, consisting of:

- Device Layer – ESP32 microcontroller, sensors, LEDs, and push button
- Cloud Layer – Firebase Realtime Database for data storage and synchronisation
- Presentation Layer – Web dashboard and Telegram notification system

This architecture enables efficient communication between devices and provides reliable real-time monitoring of the toilet water availability system.

6.1.1 Physical Installation and Real-World Deployment

The final phase of the implementation involved the physical deployment of the FlowSense system at the designated site to evaluate its performance in a real-

world environment. The system was installed within the plumbing area of the facility. The ESP32 microcontroller, housed in a protective enclosure, was securely mounted to ensure stability and protection from environmental factors.

The XKC-Y25 non-contact liquid level sensor was strategically attached to the external surface of the water pipe. This placement allows the sensor to detect water presence through the pipe wall without direct fluid contact, preventing corrosion and ensuring long-term durability. Additionally, the raindrop sensor and the manual reporting push button were positioned to be easily accessible for monitoring and user interaction. This real-world setup confirms the system's ability to operate in a practical setting, successfully bridging the gap between a lab prototype and a functional monitoring solution.



Figure 6.1.1.1 Physical installation and deployment of the FlowSense system on-site

6.2. Testing approach/Strategy and Design

To ensure that the FlowSense system operates correctly and reliably, a testing approach consisting of component testing and system integration testing was conducted. The purpose of the testing process was to verify that each hardware component, software function, and communication system operates according to the design specifications.

6.2.1 Component Testing

During the component testing stage, each individual hardware component was tested separately to verify correct functionality. The XKC-Y25 water level sensor was tested by placing it near a pipe containing water to confirm that it could accurately detect the presence of water. The raindrop sensor was tested by applying small amounts of water on its surface to ensure it could detect leakage conditions. The push button was tested to confirm that pressing the button correctly triggers the BROKEN system status. The LED indicators were also tested to ensure that each LED correctly represents the corresponding system condition:

- Green LED → Water available
- Red LED → No water supply
- Yellow LED → Fault condition

All components responded according to the expected results.

6.2.2 System Integration Testing

After confirming that each component functions correctly, full system integration testing was conducted. In this stage, the ESP32 continuously reads sensor data, processes the information based on the system logic, updates LED indicators, and transmits the data to Firebase. Several test scenarios were designed to evaluate the behaviour of the system.

Scenario	Condition	Expected Output	Actual Result	Status
XKCY25 Sensor	Water level detected	Green LED (XKC-Y25) ON, Red LED OFF, status = AVAILABLE	Same	Pass
XKC-Y25 Sensor	No water detected	Red LED (XKC-Y25) ON, status = NO WATER	Same	Pass
Raindrop Sensor	Water detected on the surface	Green LED (Raindrop) ON, Red LED OFF status = AVAILABLE	Same	Pass
Raindrop Sensor	No water	Red LED (Raindrop) ON	Same	Pass
Push Button	Button pressed by user	Yellow LED ON, Status = BROKEN + alert	Same	Pass
System Integration	Both sensors operating simultaneously	Correct LED states based on each sensor + real-time Firebase update	Same	Pass
WiFi/Firebase	Stable WiFi connection	Data successfully synced to Firebase in real-time	Same	Pass

The system was also evaluated for real-time performance. Data was successfully transmitted from the ESP32 to Firebase at regular intervals, and the web dashboard updated automatically without requiring manual refresh due to Firebase real-time synchronization. Telegram notifications were configured to trigger only when a change in system state occurs. This ensures that users receive meaningful alerts without redundant or repeated notifications.

All test results were compared with expected outputs based on the system design and flowchart. The results confirm that the FlowSense system operates successfully with high accuracy and reliability.

The system demonstrates:

- Accurate water detection and fault identification
- Stable real-time communication with cloud services
- Effective IoT integration between hardware, cloud, and application layers
- Reliable notification and monitoring system

Overall, the FlowSense Water Availability System meets all functional requirements and successfully achieves its intended objectives as an IoT-based monitoring solution.

7. Conclusion

In conclusion, the FlowSense Water Availability System successfully achieves its goal of providing a smart and efficient solution to monitor water availability in dormitory toilets. By integrating sensors, an ESP32 microcontroller, cloud-based Firebase services, and a real-time dashboard, the system is able to detect water conditions, notify users, and support faster maintenance responses. The implementation and testing results confirm that the system is reliable, accurate, and capable of real-time operation. Furthermore, the project demonstrates how IoT technology can be effectively applied to solve practical, real-world problems in shared environments. Overall, this project not only improves user convenience and facility management but also highlights the potential of digital technology in creating smarter and more sustainable living spaces.

7.1 Future Work

Several improvements can be made to further enhance the functionality, scalability, and robustness of the FlowSense system. Firstly, additional sensors such as flow rate sensors or ultrasonic sensors can be integrated to provide more precise and multi-level water monitoring instead of binary detection. Secondly, the system can be expanded into a mobile application with push notifications to improve accessibility and user engagement beyond the web dashboard. Thirdly, implementing data analytics and historical data tracking in Firebase would allow predictive maintenance, enabling early detection of recurring issues.

Moreover, improving network reliability through offline data buffering or alternative communication protocols (e.g., MQTT) would enhance system resilience in unstable WiFi environments. The system can also be scaled to support multiple buildings or campuses by implementing a more structured database and user management system. Finally, incorporating energy-efficient designs such as solar power or optimized power consumption modes would make the system more sustainable for long-term deployment. These enhancements would transform the current prototype into a more advanced, scalable, and industry-ready IoT solution.

8. References

- Chin, J. Y., Tan, Y. C., & Wahid, K. A. (2022). *Smart toilet monitoring system using IoT and sensor network for public facilities. International Journal of Smart Sensor and Ad Hoc Networks*, 3(2), 45–51.
- Circuit Digest. (2022). *Interfacing rain sensor module with ESP32 to detect water or leakage*. Circuit Digest.
<https://circuitdigest.com/microcontroller-projects/interfacing-rain-sensor-with-esp32>
- Electronics Hub. (2022). *Non-contact water level sensor (XKC-Y26) interfacing with Arduino/ESP32*. Electronics Hub.
<https://www.electronicshub.org/non-contact-water-level-sensor-xkc-y26/>
- IoT Design Pro. (2021). *IoT-based water level monitoring using NodeMCU and Blynk*. IoT Design Pro.
<https://iotdesignpro.com/projects/iot-water-level-monitoring-using-nodemcu-and-blynk>
- Karthikeyan, T., & Babu, R. (2021). *Design and implementation of IoT based water management system using ESP32. International Research Journal of Engineering and Technology (IRJET)*, 8(5), 3025–3030.
- Raj, P., Raman, A. C., & Chatterjee, J. M. (2020). *The internet of things: Enabling technologies, platforms, and use cases*. CRC Press.
- Rani, P., & Singh, S. (2020). *Smart water monitoring system using internet of things. International Journal of Engineering Research & Technology (IJERT)*, 9(6), 560–564.
- Sharma, N. K., & Saini, J. R. (2021). *Smart water management systems: Technologies and applications*. Springer.
- Tijani, I. A., & Singh, S. (2021). *Internet of things (IoT) projects with ESP32*. Packt Publishing.

9. Presentation Video

9.1. Timeframe

Table

Question	Start Time	End Time	Matric Number	ID Number
Q1: Implementation	0.00	1.32	MC2516116848	P4801
Q2: Testing	1.32	2.30	MC2516116784	P4803
Q3: Working in a Team	2.30	3.53	MC2516116830	P4804
Q4: Learning New Technical Skills	3.53	5.41	MC2516203254	P4802
Q5: Leadership	5.41	6.34	MC2516116784	P4803